SOFTWARE ENGINEERING · PROJECT #11

World Cup Analytics

An Elo rating engine and match predictor built over 49,547 international football matches played between 1872 and 2026.

49,547
MATCHES

60.1%
ACCURACY

91
TESTS

The dashboard: 23 World Cups, 1,068 matches, 3,028 goals

Mohammed Majeed Khan · SE23UMCS071

github.com/Majeedkhan05/world-cup-analytics

THE BRIEF

Real data, verified before a line of code

Project #11 asked for World Cup analytics. The first decision was where the data comes from — and whether it can be trusted.

The dataset

`results.csv`	**49,547**
every international match, 1872-2026	
`goalscorers.csv`	**47,914**
individual goals, scorer and minute	
`shootouts.csv`	**682**
penalty shootout outcomes	

Verified against the historical record

✓	2018	France 4-2 Croatia	Moscow
✓	2022	Argentina 3-3 France, won on pens	Lusail
✓	2014	Germany 1-0 Argentina	Rio de Janeiro

Top scorers match the record exactly too: Klose 16, Ronaldo 15, Muller 14, Fontaine 13, Pele 12.

Why this project, and why it is not a CRUD app

1. **Real data, not seeded rows**
   150 years of actual results — the analysis has something to find.
2. **A model, not just queries**
   An Elo engine and a calibrated predictor, both implemented from scratch.
3. **Measured, not asserted**
   The model reports its own accuracy on data it never saw.

ARCHITECTURE

A layered system with one strict rule

The expensive work happens once, offline. The web tier only ever reads pre-computed results.

The rule that makes it testable

Routes contain no SQL. Services contain no HTTP. The rating engine touches neither — it is pure functions over numbers, so the most important logic can be tested with no database and no web server.

Why pre-compute in the ETL

Replaying 49,547 matches takes about two seconds. Doing that per request would make every page slow; doing it once at build time makes every page instant — pages render in milliseconds.

Project structure

```
app/
  config.py    etl.py     schema.sql   db.py
  services/    elo · predictor · analytics
  routes/      pages.py   api.py
  templates/   static/
```

```
scripts/tune_elo.py   hyper-parameter search
tests/                91 tests
docs/                 SRS · design · testing
data/raw/             the three source CSVs
```

STAGE 1 · DATA PIPELINE

Extract, transform, load

One command rebuilds the entire database from raw CSV in about five seconds.

1. **Extract**
   Read the three CSVs with pandas.

2. **Transform**
   Drop unplayed fixtures, cast types, build the team dimension, derive each tournament's champion, replay 150 years through the Elo engine, calibrate and score the model.

3. **Load**
   Write every table inside a single transaction.

```
def clean_matches(results):
    df = results.dropna(
        subset=["home_score", "away_score"])
    df["year"] = df["date"].str[:4].astype(int)
    df["is_world_cup"] = (
        df["tournament"] == WORLD_CUP).astype(int)
    df = df.sort_values("date", kind="stable")
    df["id"] = df.index + 1
    return df
```

Sorting by date is not cosmetic — the Elo engine must replay matches in the order they were actually played.

The bug worth talking about: who won the 1950 World Cup?

I derived each champion as "the winner of the last match played". That works for 22 of the 23 editions — but 1950 had no final. It ended in a four-team round-robin whose last two matches were on the same day, so my code crowned Sweden instead of Uruguay. The fix breaks the tie by picking the fixture between the two highest-scoring teams of the tournament, and a regression test now asserts 1950 → Uruguay.

STAGE 2 · DATA MODEL

A normalised schema, and one view that pays for itself

Nine tables, plain SQL, no ORM — because in this project the queries are the analysis.

Tables

- teams
- matches
- goals
- shootouts
- tournaments
- elo_ratings
- elo_history
- draw_calibration
- model_metrics

matches also stores home_elo and away_elo — each side's rating as it stood before kick-off. That single design choice is what makes honest evaluation possible later.

The team_matches view

```
CREATE VIEW team_matches AS
  SELECT id, home_team_id AS team_id,
         away_team_id AS opponent_id,
         home_score AS gf, away_score AS ga
  FROM matches
  UNION ALL          -- same match, other side
  SELECT id, away_team_id, home_team_id,
         away_score, home_score FROM matches;
```

A match is stored once, but almost every question is asked from one team's point of view.

The view writes each match twice — once per side — so "how did team X do?" collapses to a single WHERE clause. That is why the whole per-team record is one short query instead of a page of Python:

```
SELECT COUNT(*)     AS played,
       SUM(gf > ga) AS won,
       SUM(gf = ga) AS drawn,
       SUM(gf < ga) AS lost
FROM team_matches WHERE team_id = ?
```

STAGE 3 · THE MODEL

The Elo rating engine

Every nation starts at 1500. All 49,547 matches are replayed in date order, and each result nudges both ratings.

$$E = 1 / (1 + 10^{(-(Ra + H - Rb)/400)}) \qquad R' = R + K \times w \times m \times (S - E)$$

E is what the ratings expected. S is what happened: 1 win, 0.5 draw, 0 loss. The rating moves by the difference between them — so a team gains points only by doing better than predicted.

Zero-sum

Whatever the winner gains, the loser loses. Ratings never inflate.

The 400 defines Elo

A 400-point gap means the stronger side is expected to take ten times as many points. A unit test asserts exactly this.

Three multipliers

K sets the learning rate; w weights a World Cup at 2.0 and a friendly at 0.7; m is sqrt(goal difference), capped.

```
def update_pair(r_home, r_away, hs, aws,
                tournament, neutral):
    adv = 0 if neutral else ELO_HOME_ADVANTAGE
    E = expected_score(r_home, r_away, adv)
    S = outcome(hs, aws)        # 1 / 0.5 / 0
    k = (ELO_K_BASE
         * tournament_weight(tournament)
         * margin_factor(hs, aws))
    delta = k * (S - E)
    return r_home + delta, r_away - delta
```

Why the margin is capped

A 9-0 says little more about a team's strength than a 5-0, but uncapped it would swing the rating violently. The cap keeps big wins meaningful without letting one freak result distort a rating.

Germany's rating peaks on 13 July 2014 — the day they won the World Cup.

STAGE 4 · PREDICTION

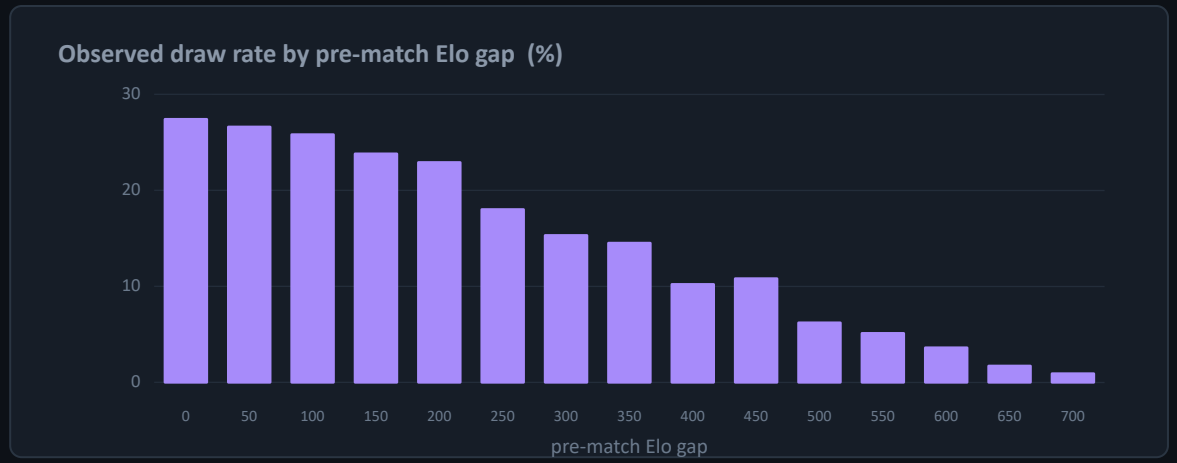
One rating gap into three probabilities

Elo gives a single number that blends wins and draws. Splitting it needs a draw model — so I measured one instead of assuming it.

Measured, not assumed

Because every match stores the ratings that existed before kick-off, I could bucket all 49,547 matches by that gap and count the real draw rate in each bucket.

The result is on the right: evenly matched teams draw 27% of the time, badly mismatched ones under 2%. A fixed 25% would have been measurably worse.



The algebra

Since E = P(win) + ½ · P(draw)

```
P(draw)   =  measured rate for this gap
P(win)    =  E − P(draw) / 2
P(loss)   =  1 − P(win) − P(draw)
```

At extreme gaps P(loss) can go negative, so values are clamped and renormalised — a unit test drives a 1,400-point gap through it.

```python
def probabilities(r_a, r_b, calib, neutral):
    adv = 0 if neutral else ELO_HOME_ADVANTAGE
    E   = expected_score(r_a, r_b, adv)
    gap = (r_a + adv) - r_b

    p_draw = draw_probability(gap, calib)
    p_a    = E - p_draw / 2
    p_b    = 1 - p_a - p_draw

    # clamp, then renormalise
    total = p_a + p_draw + p_b
    return p_a/total, p_draw/total, p_b/total
```

STAGE 5 · RIGOUR

Tuned by search, judged on data it never saw

The two things that separate a model you can defend from a model you merely built.

Constants were searched, not guessed

scripts/tune_elo.py sweeps 96 combinations of K, home advantage and margin cap. For each one it replays the entire match history and scores it on held-out matches.

```
K = 25   home = 65   margin cap = 3.0
-> best Brier of 96 combinations
```

My first hand-picked K produced wildly overconfident forecasts. The search fixed that.

A clean train / test split

Fit the draw model on matches before 2015 only

Score on the 11,130 matches played since

Never use a rating from after the match being predicted

Every prediction uses only ratings that existed before kick-off, so this is a walk-forward evaluation — not hindsight.

Accuracy on 11,130 unseen matches

0.516

BRIER SCORE

Guessing scores 0.667. Lower is better.

11,130

TEST MATCHES

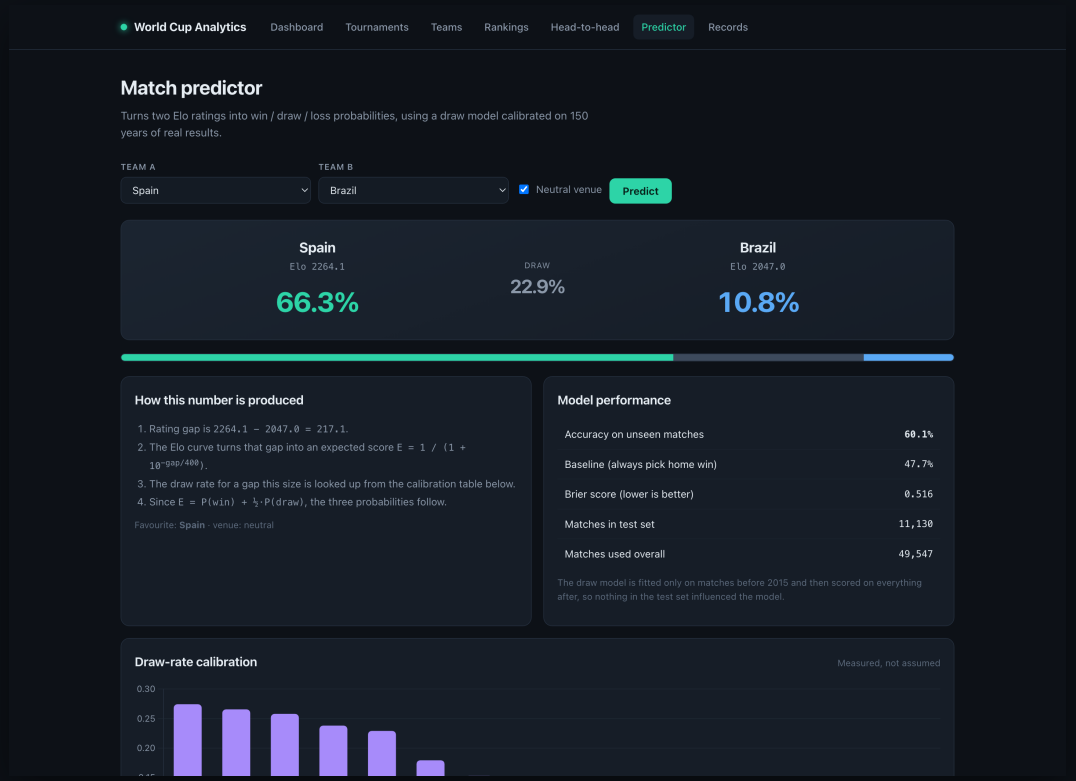
Large enough not to be noise.

Is 60% good? For three-way football outcomes, yes — professional models sit in the mid-to-high 50s. A model claiming 90% would be a model with a bug.

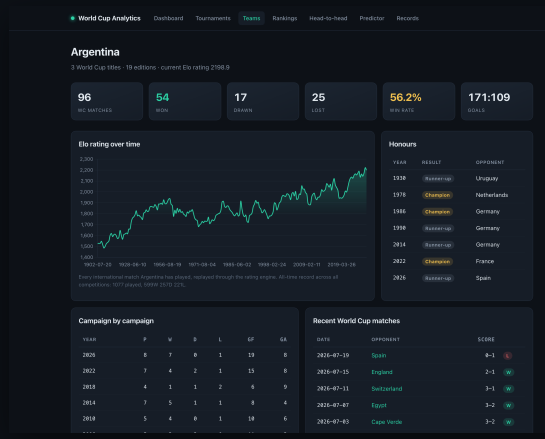
THE PRODUCT

Eight pages and a JSON API over one service layer

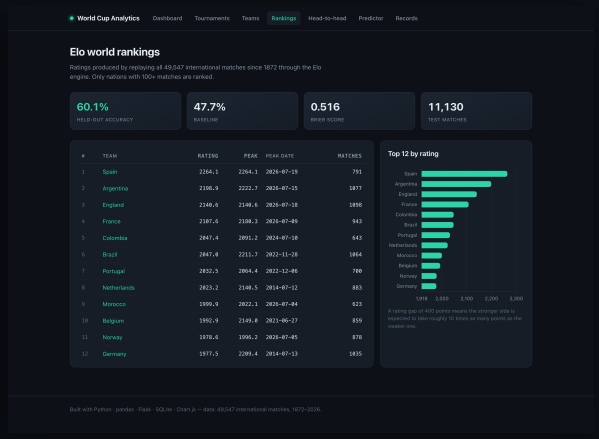
Server-rendered Jinja2 with Chart.js. No build step, no bundler, no frontend framework.



The predictor shows its own working — and its own accuracy



Team explorer — 1,077 matches replayed



Elo world rankings

Same services, two front doors

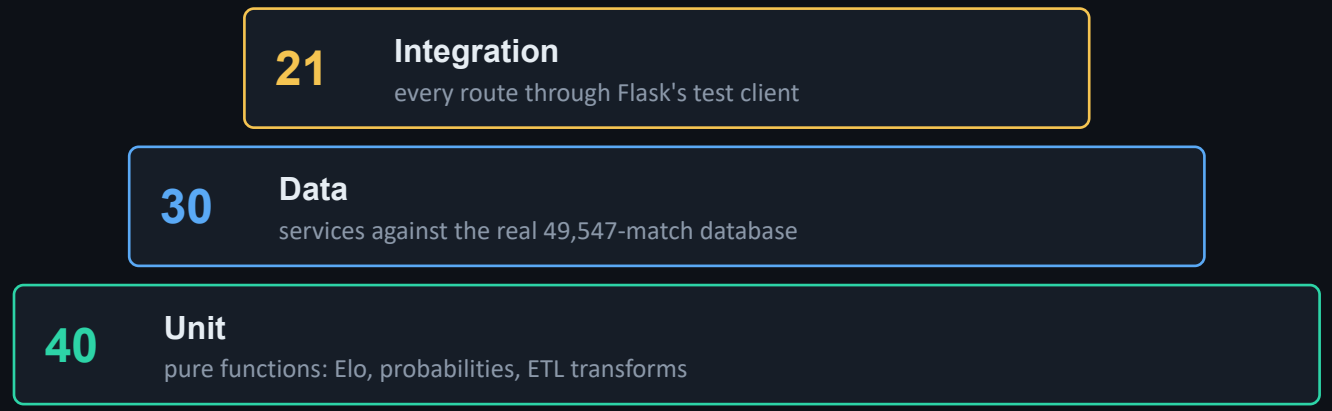
Every figure on the site is also machine-readable — 14 JSON endpoints backed by the identical service functions the pages use.

```
$ curl '.../api/predict?a=Spain&b=Brazil'
{"pct_a": 66.3, "pct_draw": 22.9,
 "pct_b": 10.8, "favourite": "Spain"}
```

QUALITY

91 tests, shaped by the fact that the core logic is pure

The Elo maths and the probability functions take numbers and return numbers — so the code most likely to be wrong is also the cheapest to test.



On a fresh checkout, before the database exists, 40 tests still pass and 51 skip with a clear message.

Properties, not examples

Rather than asserting hard-coded outputs, the tests assert the properties the model must have:

- A 400-point gap gives exactly 10:1 odds
- Rating changes are always zero-sum
- Both sides' expectations sum to 1
- Probabilities stay valid at a 1,400-point gap
- Pre-match ratings are captured before the update

The test that matters most

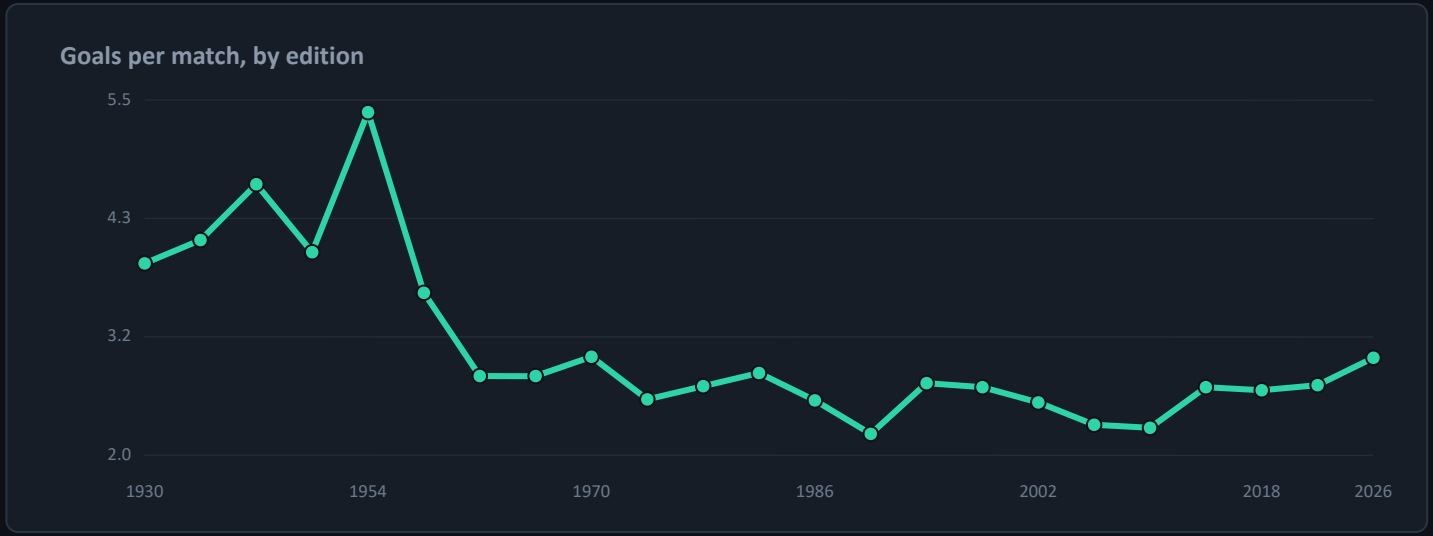
If pre-match ratings were recorded after the update instead of before, every accuracy figure in this deck would be inflated by hindsight — and nothing would look broken. One test pins that ordering in place.

```
def test_pre_match_ratings_are_recorded
        _before_the_update(self):
    engine = EloEngine()
    engine.process(match)
    assert engine.pre_match[match["id"]] == (1500, 1500)
    assert engine.rating("A") > 1500
```

RESULTS

What the data actually says

Four findings that fell out of the analysis, each reproducible from the database.



5.38

1954 is the highest-scoring World Cup ever. No edition since 1962 has reached 3.0.

61.2%

Host nations win 61.2% of their matches; visiting nations win 37.4%.

+32%

The second half produces 32% more goals than the first — 1,595 against 1,207. The closing 15 minutes are the single most productive window.

27% → 2%

Draw rate collapses as the rating gap widens. This is why the predictor needs a calibrated draw model, not a fixed 25%.

HONEST CLOSE

What it does not do, and what comes next

Volunteering the limits is stronger than being caught by them.

Limitations

- Elo uses only results — no squads, injuries or rest. That is the real accuracy ceiling.
- West Germany and Germany stay separate, as the source records them.
- The source has no match-stage labels, so group and knockout football cannot be split.
- Draw calibration is global, not per-era, though draw rates have drifted since 1872.

With more time

1. **Match-stage labels**
   separate group from knockout football
2. **Era-adjusted ratings**
   a 2000 rating today is not a 2000 rating in 1950
3. **Tournament simulator**
   run the bracket 10,000 times for each team's title odds

49,547	23	337	60.1%	91	~5s
MATCHES ANALYSED	WORLD CUPS	NATIONS RATED	HELD-OUT ACCURACY	TESTS PASSING	FULL REBUILD

github.com/Majeedkhan05/world-cup-analytics

Python · pandas · Flask · SQLite · Chart.js · pytest